



南開大學
Nankai University

MySQL 数据库综合操作实验

(2024~2025 学年第二学期)

实验名称: MySQL 数据库综合操作实验

学 院: 软件学院

姓 名: 郭威威

学 号: 2414308

指导老师: 李旭东

2024~2025 春季学期

使用 L^AT_EX 撰写于 2025 年 3 月 27 日

目录

1	实验目的	2
2	实验环境	3
2.1	操作系统	3
2.2	软件版本	3
3	实验步骤	4
3.1	Python 数据库连接	4
3.1.1	初始连接问题	4
3.1.2	正确连接配置	5
3.2	数据库表设计 (DDL)	7
3.2.1	创建基础表	7
3.2.2	添加外键约束	9
3.2.3	数据完整性管理	9
3.2.4	删除	11
3.3	数据操作 (DML)	12
3.3.1	数据导入	12
3.3.2	数据检索	13
3.3.3	数据更新	13
3.3.4	数据删除	14
4	实验结果	16
4.1	数据库操作验证	16
4.1.1	表结构验证	16
4.1.2	数据完整性验证	16
5	实验总结	18
5.1	技术收获	18
5.2	典型问题处理	18
5.3	实验启示	18

实验一 实验目的

掌握 Python 与 MySQL 数据库的交互方法，熟练进行数据库表结构设计、数据操作及复杂查询，理解外键约束的应用与数据完整性管理，提升数据库综合操作能力。

实验二 实验环境

2.1 操作系统

Windows 11 Pro 22H2

2.2 软件版本

软件名称	版本号
MySQL	8.0.41
MySQL Workbench	8.0.36
Python	3.11
PyMySQL	1.1.1

表 2.1: 实验软件版本信息

实验三 实验步骤

3.1 Python 数据库连接

3.1.1 初始连接问题

首次使用错误数据库导致异常：

```
1 pymysql.err.ProgrammingError: (1146, "Table 'sys.teacher' doesn't ...  
    exist")
```

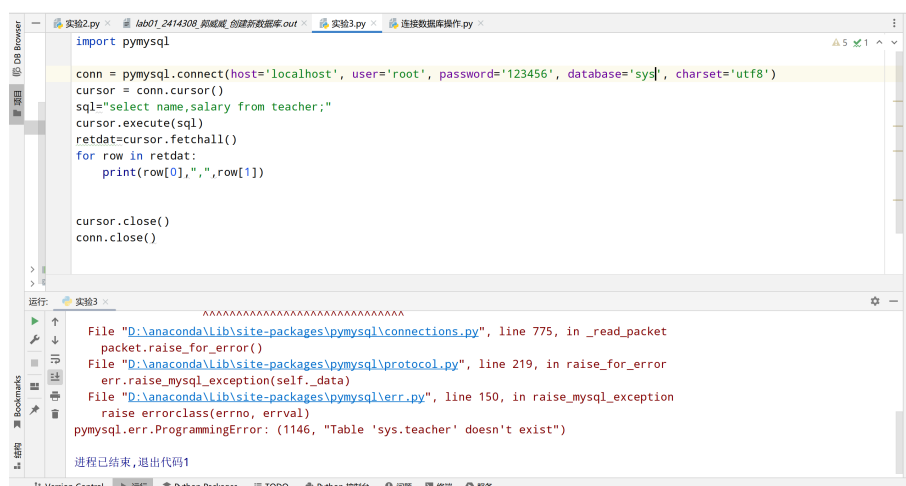


图 1 Python 数据库连接错误截图

修改数据库连接参数后仍因用户名错误导致：

```
1 pymysql.err.OperationalError: (1045, "Access denied for user '...  
    myuserco'@'localhost'")
```

```
File "D:\anaconda\lib\site-packages\pymysql\connections.py", line 775, in _read_packet
    packet.raise_for_error()
File "D:\anaconda\lib\site-packages\pymysql\protocol.py", line 219, in raise_for_error
    err.raise_mysql_exception(self._data)
File "D:\anaconda\lib\site-packages\pymysql\err.py", line 150, in raise_mysql_exception
    raise errorclass(errno, errval)
pymysql.err.OperationalError: (1045, "Access denied for user 'myuserco'@'localhost' (using password: YES)")

进程已结束,退出代码1
```



图 2、3 MySQL 连接配置错误截图

再次运行后发现原数据库中没有 teacher 表:

```
1 pymysql.err.ProgrammingError: (1146, "Table 'dbsc1ab2025.teacher' ...
    doesn't exist")
```

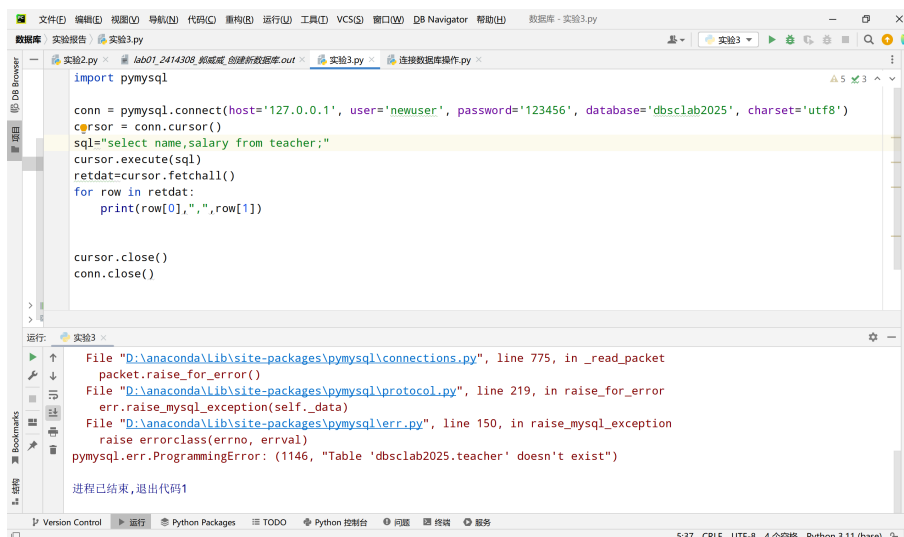


图 4 数据库表不存在错误截图

3.1.2 正确连接配置

最终使用正确参数连接成功: 改成 instructor 表后成功显示数据

```
1 import pymysql
2 conn = pymysql.connect(
3     host='127.0.0.1',
4     user='newuser',
5     password='123456',
6     database='dbsclab2025',
7     charset='utf8'
8 )
9 cursor = conn.cursor()
10 cursor.execute("SELECT name, salary FROM instructor")
11 for row in cursor.fetchall():
12     print(f"{row[0]}, {row[1]}")
13 cursor.close()
14 conn.close()
```

运行结果显示：

```
1 Lembr, 32241.56
2 Bawa, 72140.88
3 Yazdi, 98333.65
4 Wieland, 124651.41
5 DAgostino, 59706.49
6 Liley, 90891.69
7 Kean, 35023.18
8 Atanassov, 84982.92
9 Moreira, 71351.42
```

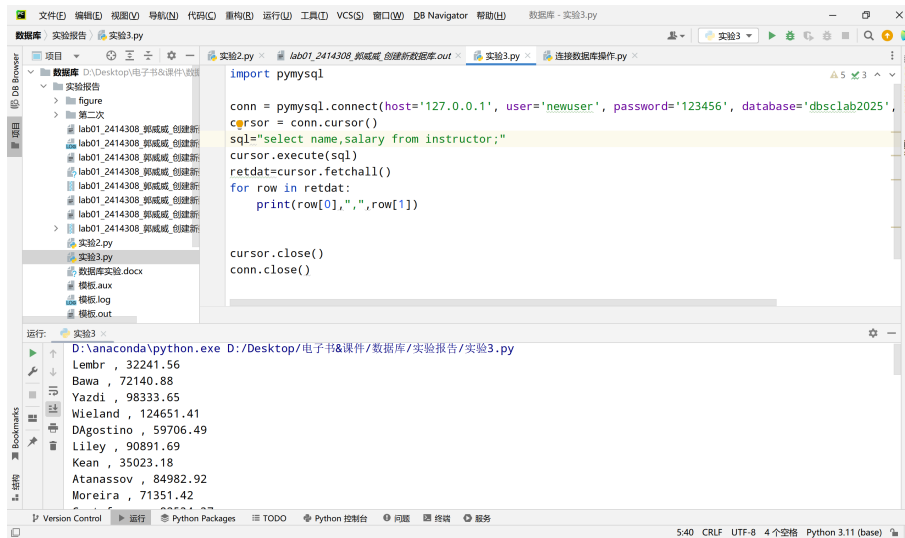


图 5 Python 连接成功结果截图

3.2 数据库表设计（DDL）

3.2.1 创建基础表

```
1 CREATE TABLE department01 (  
2     deptname VARCHAR(255) PRIMARY KEY,  
3     location VARCHAR(255)  
4 );  
5  
6 CREATE TABLE student01 (  
7     ID INT PRIMARY KEY,  
8     Name VARCHAR(255) ,  
9     Birthplace VARCHAR(255)  
10 );
```

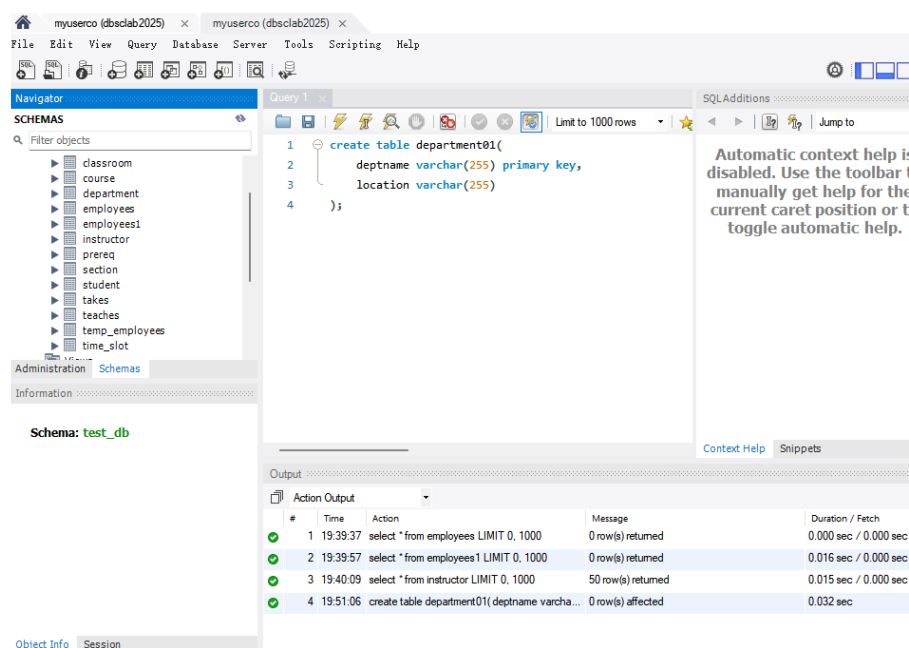



图 6 创建 department01 表操作截图

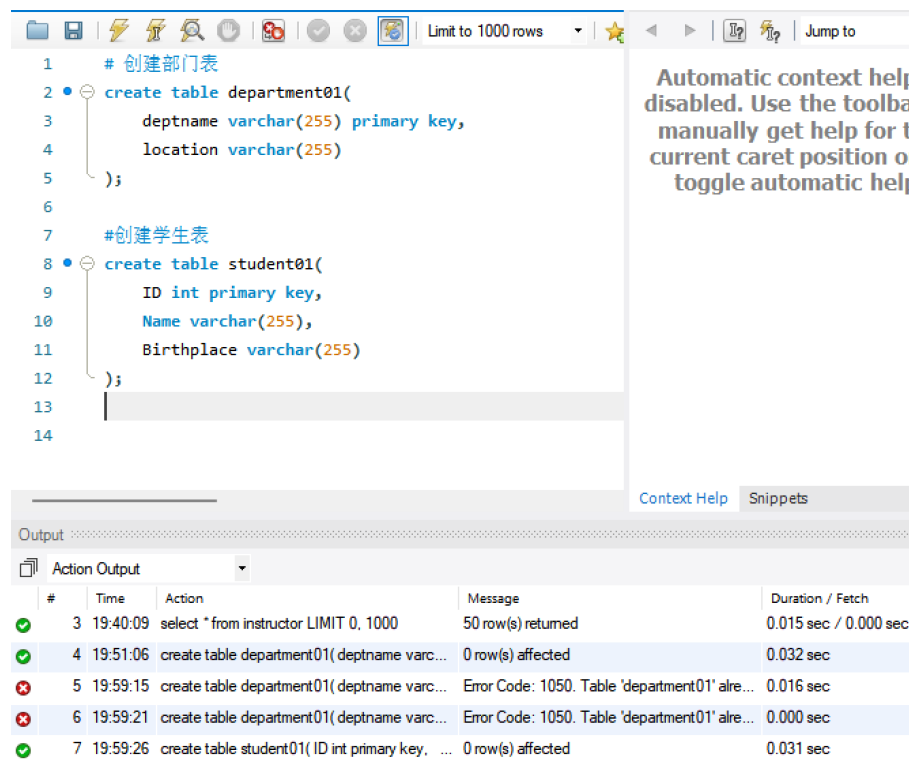


图 7 创建 student01 表操作截图

3.2.2 添加外键约束

增加外键时出错，单行运行后成功运行

```
1 ALTER TABLE student01
2 ADD COLUMN deptname VARCHAR(255),
3 ADD FOREIGN KEY (deptname) REFERENCES department01(deptname);
```

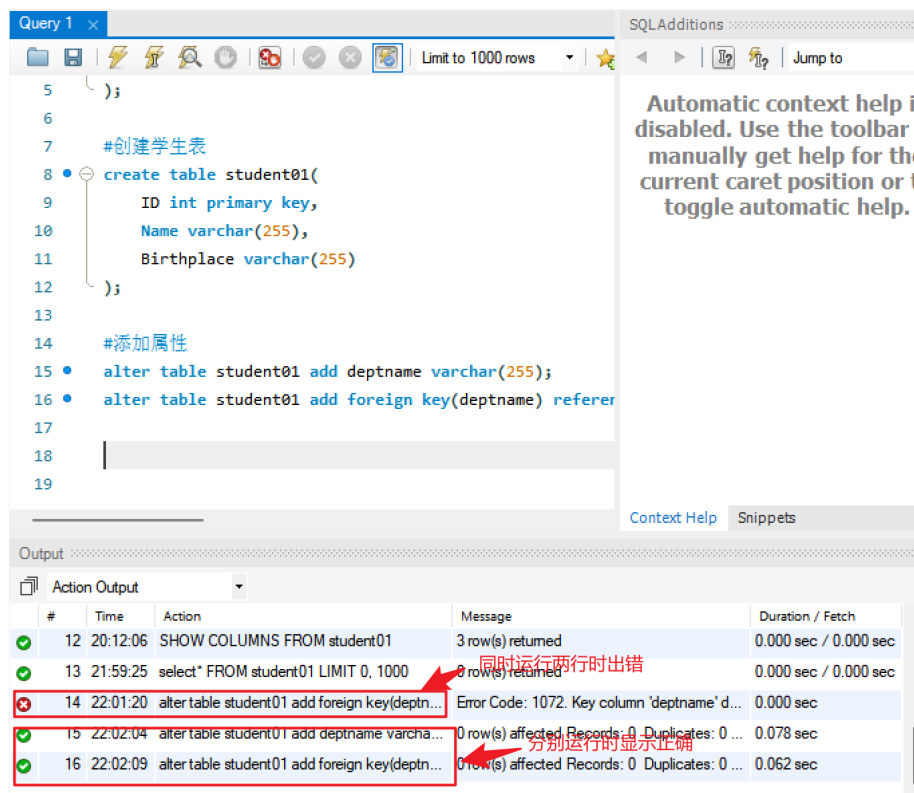


图 8 添加外键约束截图

3.2.3 数据完整性管理

通过外键约束实现数据依赖，插入数据时需保证外键存在：向 student01 插入数据出错，发现外键不对应引起增加相应外键

```
1 INSERT INTO department01(deptname) VALUES ('cs'), ('ee'), ('me'), ...  
   ('ce');  
2 INSERT INTO student01 (ID, Name, Birthplace, deptname) VALUES  
3 (1, 'alice', 'new york', 'cs'),  
4 (2, 'bob', 'chicago', 'ee'),  
5 (3, 'charlie', 'los angeles', 'me'),  
6 (4, 'david', 'houston', 'ce'),  
7 (5, 'eve', 'phoenix', 'cs');
```

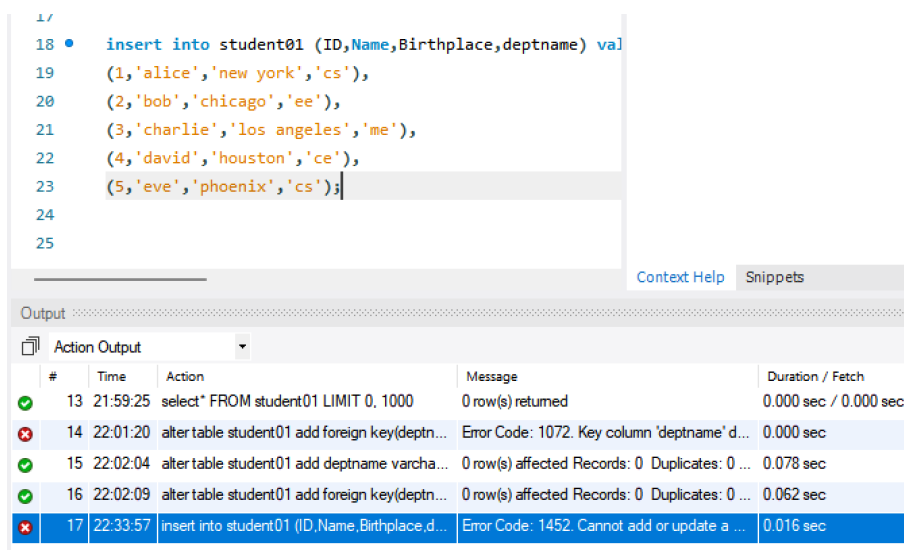


图 9 部门表数据插入截图

增加相应外键后成功:

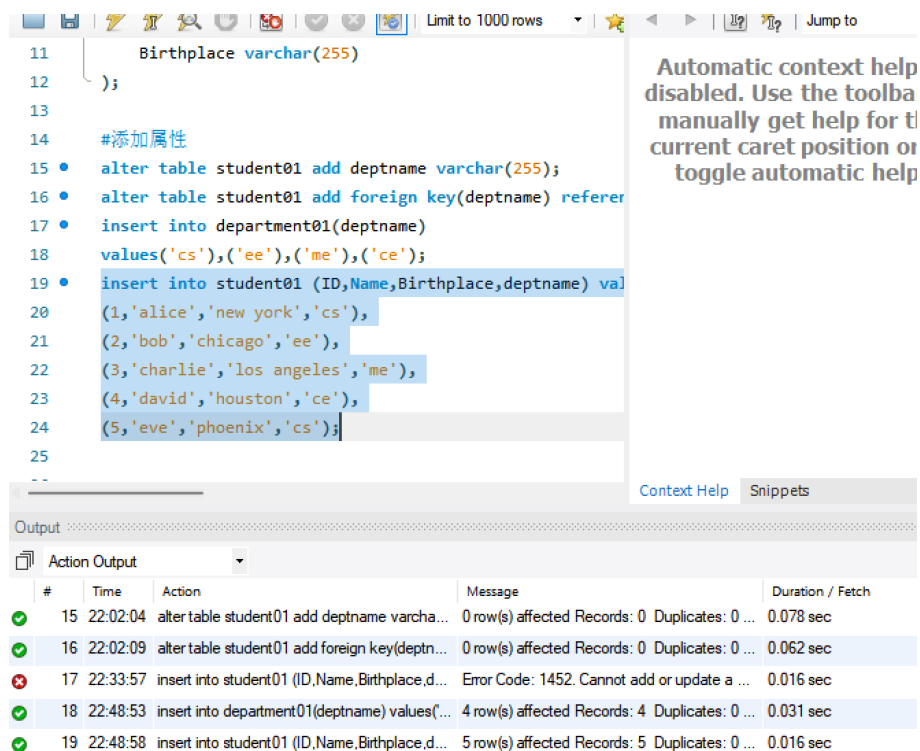


图 10 学生表数据插入截图

3.2.4 删除

删除表失败，发现 student01 引用了 department01 的外键

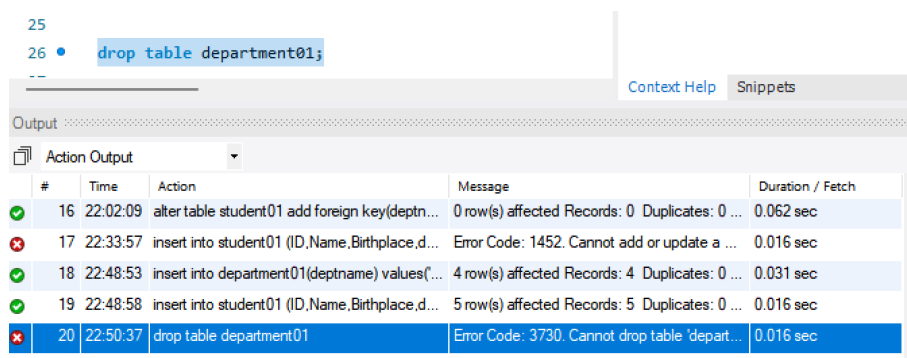


图 11 数据删除操作截图 1

将 student01 表中对 department01 表的外键引用删除，删除后成功

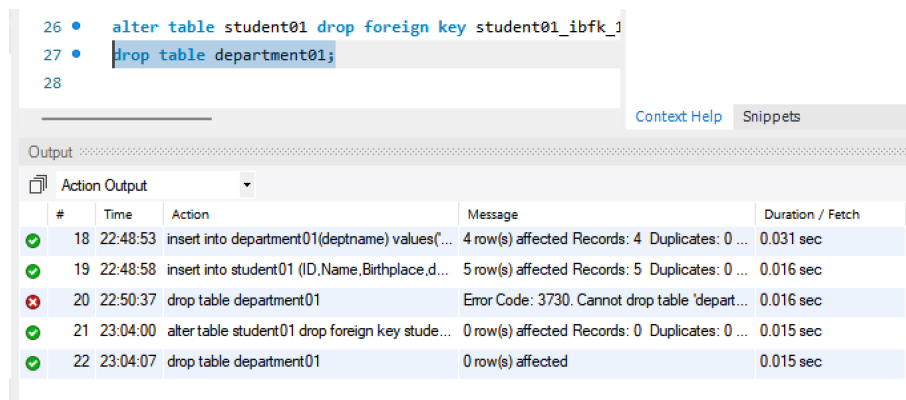


图 12 数据删除操作截图 2

3.3 数据操作（DML）

3.3.1 数据导入

注意导入的数据之间一一对应

```
1 INSERT INTO student01(ID, Name, Birthplace, deptname)
2 SELECT ID, name, NULL, dept_name FROM student;
```

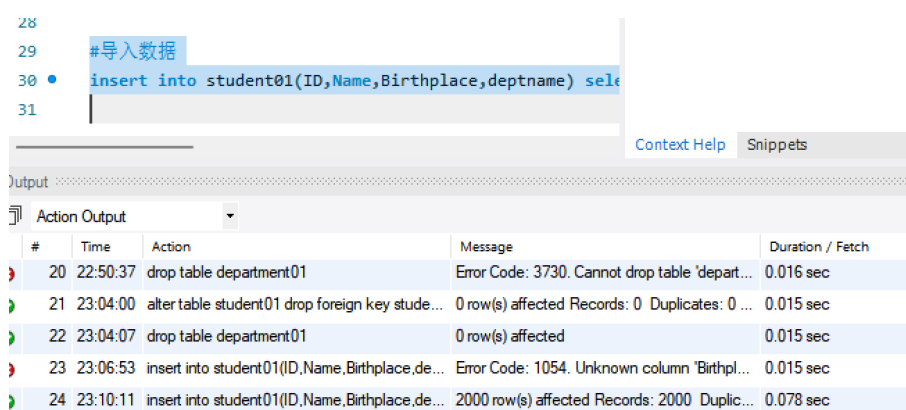


图 13 数据导入操作截图

3.3.2 数据检索

```
1 SELECT Name, deptname FROM student01;
```

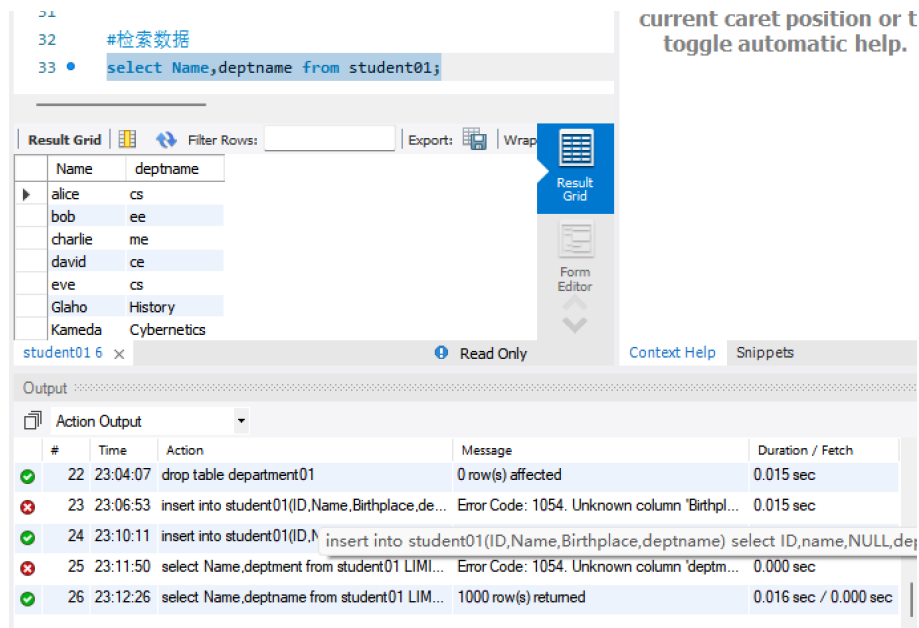


图 14 数据检索操作截图

3.3.3 数据更新

```
1 UPDATE student01 SET Birthplace='Unknown';
```

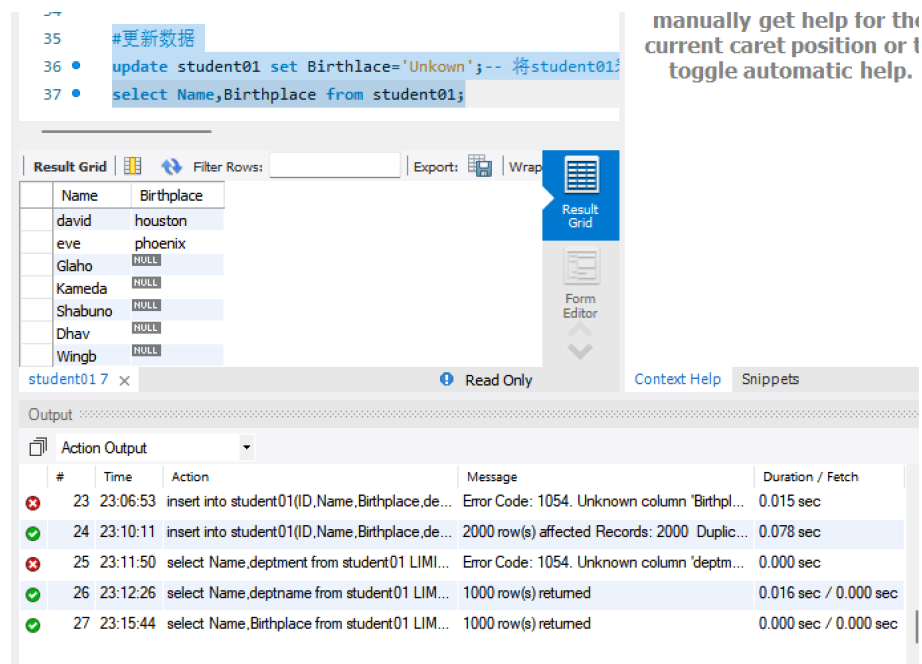


图 15 数据更新操作截图

3.3.4 数据删除

```
1 DELETE FROM student01 WHERE deptname="Comp.Sci.";
```

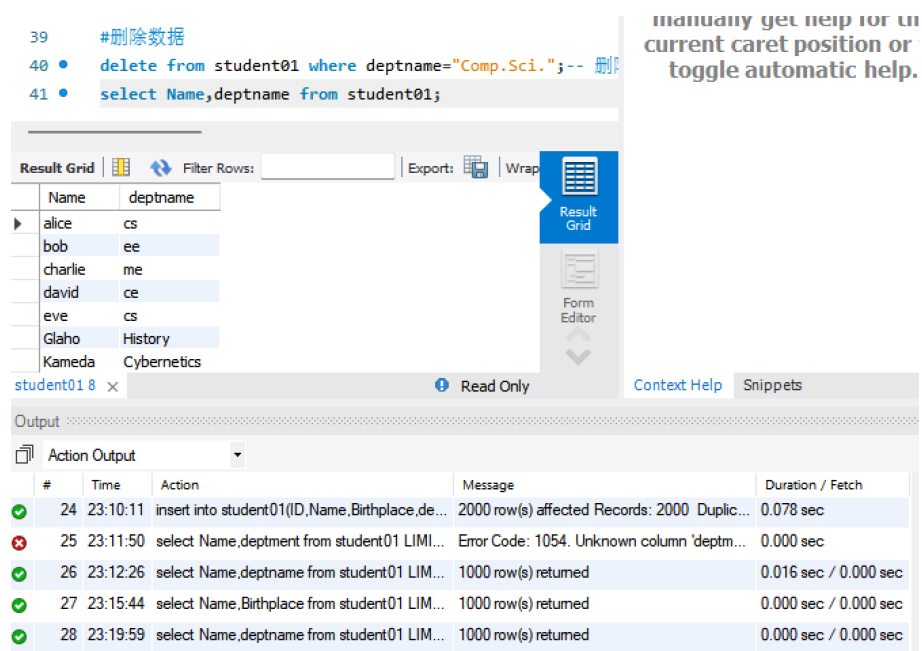


图 16 数据删除操作截图

实验四 实验结果

4.1 数据库操作验证

4.1.1 表结构验证

```
1 SHOW CREATE TABLE student01;
```

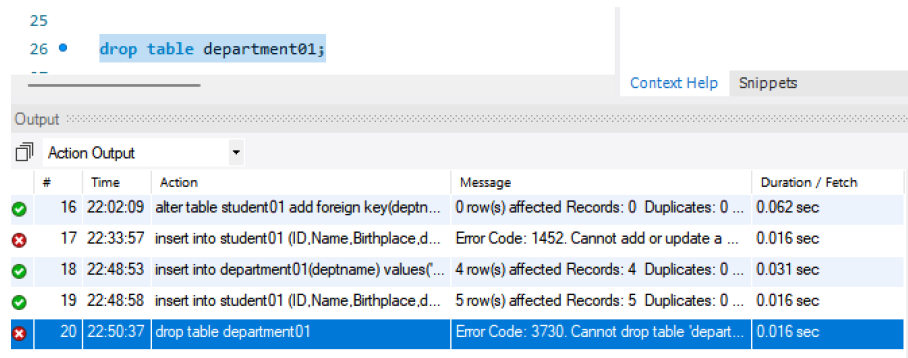
输出显示表结构包含外键约束：

```
1 CREATE TABLE student01 (  
2   ID int NOT NULL,  
3   Name varchar(255) DEFAULT NULL,  
4   Birthplace varchar(255) DEFAULT NULL,  
5   deptname varchar(255) DEFAULT NULL,  
6   PRIMARY KEY (ID) ,  
7   KEY student01_ibfk_1 (deptname) ,  
8   CONSTRAINT student01_ibfk_1 FOREIGN KEY (deptname) REFERENCES ...  
   department01 (deptname)  
9 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
```

4.1.2 数据完整性验证

删除部门表前需先删除外键约束：

```
1 ALTER TABLE student01 DROP FOREIGN KEY student01_ibfk_1;  
2 DROP TABLE department01;
```



#	Time	Action	Message	Duration / Fetch
16	22:02:09	alter table student01 add foreign key(deptn...	0 row(s) affected Records: 0 Duplicates: 0 ...	0.062 sec
17	22:33:57	insert into student01 (ID,Name,Birthplace,d...	Error Code: 1452. Cannot add or update a ...	0.016 sec
18	22:48:53	insert into department01(deptname) values('...	4 row(s) affected Records: 4 Duplicates: 0 ...	0.031 sec
19	22:48:58	insert into student01 (ID,Name,Birthplace,d...	5 row(s) affected Records: 5 Duplicates: 0 ...	0.016 sec
20	22:50:37	drop table department01	Error Code: 3730. Cannot drop table 'depart...	0.016 sec

图 11 数据完整性验证操作截图

实验五 实验总结

5.1 技术收获

1. 掌握 Python 与 MySQL 的异常处理机制 2. 深入理解外键约束对数据完整性的保障作用 3. 熟练运用 DDL 和 DML 语句进行数据库操作 4. 学会通过错误信息定位和解决数据库操作问题

5.2 典型问题处理

1. 外键冲突问题：插入数据时外键不存在导致错误，通过先插入主表数据解决。
2. 表依赖问题：删除表时存在外键依赖，通过先删除外键约束解决。
3. 字段拼写错误：更新语句中字段名拼写错误导致异常，通过仔细检查 SQL 语句修正。

5.3 实验启示

通过本次实验深刻体会到：- 数据库设计中数据完整性约束的重要性 - 开发过程中异常处理的必要性 - 数据库操作需严格遵循事务处理原则 - 版本控制工具在代码管理中的重要性